NumPy Practical Questions

1. Create and Access an Array

• Create a NumPy array containing 10, 20, 30, 40, 50.

• Print the complete array, first element, last element, change 30 to 100, and print the updated array.

2. Array Slicing

• Given a = np.array([10, 20, 30, 40, 50, 60, 70]), print the first 3 elements, elements from index 2 to 5, last 3 elements, every second element, and the reverse of the array.

3. 2D Array Indexing

• Create the matrix [[1,2,3],[4,5,6],[7,8,9]]. Print 5, the first row, last row, first column, and 8. Change 5 to 50.

4. Array Attributes

• Create a 2D array with 3 rows and 4 columns. Print ndim, shape, size, and dtype. Verify the size manually.

5. zeros(), ones() and empty()

• Create an array of 5 zeros, an array of 5 ones, a 3×3 matrix of zeros, a 2×4 matrix of ones, and a 3×3 array using empty().

6. arange()

• Using np.arange(), create numbers from 1 to 10, even numbers from 2 to 20, odd numbers from 1 to 19, and numbers from 10 down to 1.

7. linspace()

• Use np.linspace() to generate 5 equally spaced numbers between 0 and 10, 10 numbers between 1 and 100, and 6 numbers between 10 and 20.

8. Sorting an Array

• Given a = np.array([45, 12, 78, 3, 56, 23, 89]), sort it in ascending order. Verify whether the original array changed and store the sorted result separately.

9. Concatenate Two 1D Arrays

• Given a = np.array([1,2,3]) and b = np.array([4,5,6]), concatenate them to produce [1 2 3 4 5 6]. Then concatenate three 1D arrays.

10. Concatenate 2D Arrays

• Given a = [[1,2],[3,4]] and b = [[5,6]], use np.concatenate() to produce [[1,2],[3,4],[5,6]]. Explain what axis=0 does.

11. Analyze a 3D Array

• Create a = np.arange(24).reshape(3,2,4). Find ndim, shape, and size. Explain what (3,2,4) represents.

12. Reshape an Array

• Create a = np.arange(12) and reshape it into 2×6, 3×4, 4×3, 1×12, and 12×1. Do not change the total number of elements.

13. 1D to Row and Column

• Given a = np.array([10,20,30,40,50]), use np.newaxis to create a row vector with shape (1,5) and a column vector with shape (5,1).

14. expand_dims()

• Given a = np.array([1,2,3,4]), use np.expand_dims() to create arrays with shapes (1,4) and (4,1).

15. Advanced Slicing

• Given a = np.arange(1,21), print the first 5 elements, last 5 elements, elements from index 5 to 14, values at even indices, values at odd indices, and the reverse.

16. Boolean Indexing

• Given a = np.array([10,25,30,45,50,65,70]), print values greater than 40, less than 40, even values, odd values, and values between 30 and 60.

17. Boolean Indexing in 2D

• Create [[1,2,3,4],[5,6,7,8],[9,10,11,12]]. Using Boolean indexing, print values greater than 5, even values, values between 3 and 10, and values less than 5. Use & and | where required.

18. np.nonzero()

• Given a = [[10,20,30],[40,50,60],[70,80,90]], use np.nonzero() to find positions of values greater than 50, equal to 20, and even values. Print row and column coordinates.

19. Practical np.nonzero() Problem

• Given marks = [[45,78,32],[90,56,40],[67,88,29]], find coordinates of students who scored more than 60 and print the marks at those positions.

20. vstack() and hstack()

• Given a = [[1,2],[3,4]] and b = [[5,6],[7,8]], perform vertical stacking and horizontal stacking. Write the shape of both resulting arrays.

21. hsplit()

• Create a = np.arange(1,25).reshape(2,12). Use np.hsplit() to divide it into 3 equal parts, 4 equal parts, and split after columns 3 and 8. Print the resulting arrays and shapes.

22. View vs Copy

• Given a = np.array([10,20,30,40,50]), create b = a[1:4]. Change b[0] to 100 and check whether a also changes. Explain why.

23. Independent Copy

• Given a = np.array([10,20,30,40,50]), create an independent copy using .copy(). Change an element of the copy and prove that the original remains unchanged.